

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology and Engineering

Devang Patel Institute of Advance Technology and Research

Department of Information Technology

Big Data Analytics (CSUE301)

ID: 24DIT050

Practical 3

MapReduce Word Count using Hadoop

1. OBJECTIVE

The objective of this practical is to implement a MapReduce Word Count application using Hadoop. It helps in understanding how Mapper and Reducer components process large text datasets, how data is stored in HDFS, and how MapReduce can be used for distributed data processing and word frequency analysis.

2. AIM

MapReduce Word Count using Hadoop HDFS.

3. PROBLEM DEFINITION

A digital publishing company stores thousands of news articles and blog posts in a Hadoop Distributed File System (HDFS). The editorial team wants to identify the most frequently used words across their content to understand trending topics and optimize content recommendations.

As a Big Data Engineer, the objective is to develop a MapReduce application that processes text data and generates word frequency statistics. The application uses HDFS for distributed storage and MapReduce for distributed processing.

4. Tasks:

- 1. Configure the Hadoop environment and upload the given text dataset to HDFS.**

```
shalvi@LAPTOP-TF0D01GK: ~, X + v
shalvi@LAPTOP-TF0D01GK:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [LAPTOP-TF0D01GK]
2026-08-14 09:26:32,332 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
shalvi@LAPTOP-TF0D01GK:~$ mkdir -p ~/wordcount/input
cd ~/wordcount
nano input/news.txt
shalvi@LAPTOP-TF0D01GK:~/wordcount$ cat input/news.txt
Big data analytics is important.
Big data helps organizations analyze large data.
Hadoop is used for big data processing.
Hadoop provides distributed data processing.
MapReduce processes large data efficiently.
Big data and Hadoop are widely used.
shalvi@LAPTOP-TF0D01GK:~/wordcount$ jps
690 NameNode
836 DataNode
1366 Jps
1103 SecondaryNameNode
```

Figure 1: Verification of Java, Hadoop and Hadoop services

2. Successful MapReduce execution.

```
sharvi@LAPTOP-TF0D01GK: ~, X + v
sharvi@LAPTOP-TF0D01GK:~/wordcount$ hadoop jar wordcount.jar WordCount \
/user/$USER/wordcount/input \
/user/$USER/wordcount/output
2026-08-14 09:51:30,125 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
2026-08-14 09:51:31,169 INFO impl.MetricsConfig: Loaded properties from hadop
metrics2.properties
2026-08-14 09:51:31,357 INFO impl.MetricsSystemImpl: Scheduled Metric snapsh
ot period at 10 second(s).
2026-08-14 09:51:31,357 INFO impl.MetricsSystemImpl: JobTracker metrics syst
em started
2026-08-14 09:51:31,831 WARN mapreduce.JobResourceUploader: Hadoop command-l
ine option parsing not performed. Implement the Tool interface and execute y
our application with ToolRunner to remedy this.
2026-08-14 09:51:32,106 INFO input.FileInputFormat: Total input files to pro
cess : 1
2026-08-14 09:51:32,190 INFO mapreduce.JobSubmitter: number of splits:1
2026-08-14 09:51:32,624 INFO mapreduce.JobSubmitter: Submitting tokens for j
ob: job_local807634073_0001
2026-08-14 09:51:32,624 INFO mapreduce.JobSubmitter: Executing with tokens:
[]
2026-08-14 09:51:32,944 INFO mapreduce.Job: The url to track the job: http:/
/localhost:8080/
2026-08-14 09:51:32,945 INFO mapreduce.Job: Running job: job_local807634073_
0001
2026-08-14 09:51:32,946 INFO mapred.LocalJobRunner: OutputCommitter set in c
onfig null
2026-08-14 09:51:32,954 INFO output.PathOutputCommitterFactory: No output co
mmitter factory defined, defaulting to FileOutputCommitterFactory
2026-08-14 09:51:32,955 INFO output.FileOutputCommitter: File Output Committ
er Algorithm version is 2
2026-08-14 09:51:32,955 INFO output.FileOutputCommitter: FileOutputCommitter
skip cleanup _temporary folders under output directory:false, ignore cleanu
p failures: false
2026-08-14 09:51:32,957 INFO mapred.LocalJobRunner: OutputCommitter is org.a
pache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-08-14 09:51:33,019 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-08-14 09:51:33,019 INFO mapred.LocalJobRunner: Starting task: attempt_l
ocal807634073_0001_m_000000_0
2026-08-14 09:51:33,052 INFO output.PathOutputCommitterFactory: No output co
```

Figure 1: Successful execution of the Word Count MapReduce job on Hadoop.

3. Word frequency output.

```
sharvi@LAPTOP-TF0D01GK: ~, x + v
sharvi@LAPTOP-TF0D01GK:~/wordcount$ hdfs dfs -cat /user/$USER/wordcount/output/part-r-000000
2026-08-14 09:54:58,790 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
analytics      1
analyze 1
and      1
are      1
big      4
data     7
distributed    1
efficiently    1
for      1
hadoop  3
helps    1
important    1
is        2
large      2
mapreduce    1
organizations 1
processes    1
processing   2
provides     1
used        2
widely      1
sharvi@LAPTOP-TF0D01GK:~/wordcount$ hdfs dfs -cat /user/$USER/wordcount/output/part-r-000000 | sort -k2 -nr | head -10
2026-08-14 09:55:34,044 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
data      7
big        4
hadoop    3
used       2
processing 2
large      2
is         2
widely     1
provides   1
processes  1
```

Figure 3: Word frequency output generated by the MapReduce application.

4. Top 10 words .

```
sharvi@LAPTOP-TF0D01GK:~/wordcount$ hdfs dfs -cat /user/$USER/wordcount/output/part-r-000000 | sort -k2 -nr | head -10
2026-08-14 09:55:34,044 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
data      7
big        4
hadoop     3
used       2
processing 2
large      2
is         2
widely     1
provides   1
processes  1
```

Figure 4: Top 10 most frequently occurring words identified from the MapReduce output.

5. Evaluate the performance of the MapReduce application by executing it on datasets of different sizes and comparing the processing time and output.

```
total committed heap usage (bytes)=206878976
Shuffle Errors
      BAD_ID=0
      CONNECTION=0
      IO_ERROR=0
      WRONG_LENGTH=0
      WRONG_MAP=0
      WRONG_REDUCE=0
File Input Format Counters
      Bytes Read=248
File Output Format Counters
      Bytes Written=203

real    0m4.336s
user    0m8.297s
sys     0m1.478s
```

```
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=1240
File Output Format Counters
Bytes Written=210

real    0m4.122s
user    0m8.917s
sys      0m1.617s
shalvi@LAPTOP-TF0D01GK:~/wordcount$ hdfs dfs -rm -r /user/$USER/wordcount/ou
```

```
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=2480
File Output Format Counters
Bytes Written=224

real    0m4.001s
user    0m9.148s
sys      0m1.644s
```

```
shalvi@LAPTOP-TF0D01GK:~/wordcount$ shalvi@LAPTOP-TF0D01GK:~/wordcount$ cp in
put/news.txt input/small.txt
shalvi@LAPTOP-TF0D01GK:~/wordcount$ for i in {1..5}; do cat input/news.txt >
> input/medium.txt; done
shalvi@LAPTOP-TF0D01GK:~/wordcount$ for i in {1..10}; do cat input/news.txt
>> input/large.txt; done
shalvi@LAPTOP-TF0D01GK:~/wordcount$ wc -l -w -c input/small.txt input/medium
.txt input/large.txt
  6   36  248 input/small.txt
 60  360 2480 input/medium.txt
120  720 4960 input/large.txt
186 1116 7688 total
shalvi@LAPTOP-TF0D01GK:~/wordcount|
```

5. TOOLS / TECHNOLOGY USED

- Ubuntu Linux / WSL
- Hadoop 3.x
- HDFS
- Java JDK

- Bash Terminal
- MapReduce
- Java Compiler (javac)

6. KEY QUESTIONS / ANALYSIS ANSWERS

Q1. What is the role of Mapper and Reducer in the Word Count application?

The Mapper reads the input text, separates it into individual words, and generates intermediate key-value pairs in the form (word, 1).

The Reducer receives all values associated with the same word and adds them together to generate the final word frequency.

Example:

Mapper Output:

(data,1)

(data,1)

(data, 1)

Reducer Output:

(data, 3)

Q2. How are intermediate key-value pairs generated and processed?

The Mapper generates a pair (word, 1) for every occurrence of a word. Hadoop then performs the Shuffle and Sort process, grouping identical keys together.

For example:

(data,1)

(hadoop,1)

(data, 1)

After Shuffle and Sort:

data→[1,1]

hadoop → [1]

The Reducer sums these values:

Data2

hadoop 1

Q3. How does HDFS store and distribute the input dataset?

HDFS stores files as blocks. In a distributed Hadoop cluster, these blocks can be distributed across multiple DataNodes. The NameNode manages metadata about file locations, while DataNodes store the actual data blocks.

In this practical, Hadoop was configured as a single-node cluster, so the storage and processing occurred on the same system.

Q4. What advantages does MapReduce offer for processing large datasets?

MapReduce provides the following advantages:

- Parallel data processing
- Scalability
- Fault tolerance
- Distributed processing
- Data locality
- Ability to process very large datasets efficiently

It divides a large processing task into smaller tasks that can be executed across multiple nodes in a Hadoop cluster.

Q5. How does execution time vary with dataset size?

In this experiment, execution time remained relatively similar:

Small→26.653seconds

Medium→23.530seconds

Large → 26.551 seconds

The similar times indicate that Hadoop job startup and YARN overhead dominated the execution time because the datasets were relatively small.

For much larger datasets and a multi-node cluster, the effect of distributed and parallel processing would become more significant.

Conclusion

For small datasets, conventional sequential processing may be faster because Hadoop has job initialization and YARN container overhead. However, as the dataset grows and multiple nodes are available, MapReduce becomes more suitable because it can distribute processing across the cluster.

FINAL RESULT

The Hadoop Single-Node Cluster was successfully used to execute a Word Count MapReduce application. The Mapper generated (word, 1) pairs, while the Reducer aggregated the values to calculate word frequencies. The program was successfully tested with multiple input files, stop-word filtering, Top 10 word analysis, and different dataset sizes. Performance testing showed that for small-to-moderate datasets, Hadoop framework overhead significantly affects total execution time.